

Viva Session: #975128

SYSTEM

(interview start)

Hello, and welcome to this practice viva session for **2110211 Data Structures**.

Here is your scenario:

Scenario: The Missing `operator=`

A classmate wrote this simplified dynamic array. It compiles and seems to work — until it doesn't.

```
class IntArray {
public:
    IntArray(size_t n) : mSize(n), mData(new int[n]) { }
    ~IntArray()      { delete[] mData; }

    int&  operator[](size_t i)      { return mData[i]; }
    size_t size() const             { return mSize; }

private:
    size_t mSize;
    int*   mData;
};

int main() {
    IntArray a(5);
    for (size_t i = 0; i < a.size(); i++) a[i] = i * 10;

    IntArray b(3);
    b = a;           // line (1)
    b[0] = 999;      // line (2)

    // line (3): end of main - a and b are destroyed
}
```

Note what the class **does not** define: no copy constructor, no `operator=`.

- ...line (1) the code compiles even though the `operator=` was written. What runs instead, and what exactly does it copy? Describe the memory picture after line (1) — every pointer, every heap block.
2. After line (1), something has been **leaked**. What, exactly, and why?
 3. **Line (2):** which array's contents change? What is `a[0]` now?
 4. **Line (3):** what happens when `a` and `b` are destroyed at the end of `main`? Why is this the most serious problem in the program?
 5. **The fix:** describe a correct `operator=` — what steps must it perform, in what order? What is the standard guard at the top of it, and what should it return?
 6. **The bigger picture:** this class needs more than just `operator=` to be safe. What else, and what is this rule called? When does a class need all of them — and when does it need none?

(คำแปลภาษาไทย)

เพื่อนร่วมชั้นของคุณได้เขียนคลาส dynamic array แบบง่ายขึ้นมา โค้ดนี้สามารถ compile ผ่านและดูเหมือนจะทำงานได้ตามปกติจนกระทั่งเกิดปัญหาขึ้น (ดูโค้ดด้านบน)

โปรดสังเกตว่าคลาสนี้ **ไม่ได้**นิยาม copy constructor และ `operator=` เอาไว้

งานของคุณคือการเตรียมตัวอธิบายประเด็นต่อไปนี้:

1. **บรรทัดที่ (1):** โค้ด compile ผ่านแม้จะไม่มี `operator=` เขียนไว้ สิ่งใดถูกเรียกทำงานแทน และมัน copy อะไรบ้าง? จงอธิบายภาพของหน่วยความจำหลังจากบรรทัดที่ (1) — ทั้งตัวชี้ (pointer) และบล็อกใน heap แต่ละตัว
2. หลังบรรทัดที่ (1) มีบางอย่างเกิดการ **memory leak** สิ่งนั้นคืออะไร และเกิดขึ้นเพราะเหตุใด?
3. **บรรทัดที่ (2):** ข้อมูลใน array ของออบเจกต์ใดเปลี่ยนไป? ตอนนี้อยู่ที่ `a[0]` มีค่าเท่าใด?
4. **บรรทัดที่ (3):** เกิดอะไรขึ้นเมื่อ `a` และ `b` ถูกทำลาย (destruct) เมื่อสิ้นสุดฟังก์ชัน `main`? ทำไมปัญหานี้จึงเป็นปัญหาที่ร้ายแรงที่สุดในโปรแกรม?
5. **การแก้ไข (The fix):** จงอธิบายการเขียน `operator=` ที่ถูกต้อง —

เหบลอดภย สงนนคอะเร และภณเรยกวอะเร? เมอเดทคลาส
จําเป็นต้องมีครบทั้งหมด และเมื่อใดที่ไม่จําเป็นต้องมีเลย?

Let's begin with the first point:

At line (1), `b = a;` compiles even though no `operator=` is defined in `IntArray`. What actually runs here, and what exactly does it copy?

1. constructor , operator[] ถูกเรียก , = default ของ cpp ถูกเรียกทำงาน โดย b จะไปชี้ที่ pointer เดียวกับ a (เกิดปัญหาตอนที่ถ้าจะเปลี่ยนค่า b , a ก็จะถูกเปลี่ยนตามไปด้วย)
2. เกิด memory leak ตรงที่ mSize b ไม่เท่ากับ a ละก็ยังไม่ได้คืนที่ mData เก่า ตอนสร้าง constructor
3. a[0] มีค่า [999,10,20,30,40]
4. ข้อมูลจะหายไปหมด
5. ใช้ swap mData mCap mSize เพื่อแก้ปัญหานี้
6. อธิบายเพิ่มหน่อย

Let's look more closely at line (3).

When `a` and `b` go out of scope at the end of `main()`, each of their destructors executes `delete[] mData;`.

What happens during the execution of both destructors, and why does this cause a critical problem?

เกิดปัญหาว่า เรียก destructor ของ a แล้ว a จะถูกทำลาย ซึ่ง b ก็จะถูกทำลายไปด้วย เพราะมันชี้ไปที่เดียวกับ a
ที่นี้พอเรียก b มันดันไม่มี constructor ทำให้ program crashed

When `a` is destroyed, its destructor runs `delete[] mData;`.

Then, when `b` is destroyed right after, its destructor also runs `delete[] mData;` on that same address.

What is the term for deleting the same memory block twice, and

Now let's discuss how to write a correct `operator=` for `IntArray` .

If you were writing `IntArray& operator=(const IntArray& other)` from scratch:

1. What condition should you check at the very beginning (the guard)?
2. What are the next steps to properly copy the data without leaking memory or crashing?
3. What should the operator return?

1. เช็ค array มี mSize mCapo mData
2. ใช้ `std::swap` เพื่อกัน memory leak
3. `vector<T>&` reference ไปที่ `vector<T>` เพราะเกิดการสลับ

Consider what happens if someone writes `a = a;` (self-assignment).

If your `operator=` releases the old memory before copying the new data, what will happen in the case of `a = a;` ? How do we prevent this?

ถ้าเกิดเป็น `vector` เดียวกัน ซี่ที่ข้อมูลเดิม ไม่ต้องสลับ

Regarding the bigger picture: besides a destructor and `operator=` , what other special member function is still missing in `IntArray` to make it fully safe when copying?

What is this C++ design rule called, and in what situation does a class not need to implement any of them?

ไม่ค่อยเข้าใจ ขอตัวอย่าง

```
IntArray c = a; // initializing a new object 'c' from 'a'
```

Which function is called here to create `c` , and why does `IntArray` need a custom version of it?

ออ ต้องสร้าง custom constructor ขึ้นมา แล้วทำการ copy ข้อมูลโดยใช้ operator= เนี่ยแหละ

Thank you for your responses. That concludes our viva session for today.

Grade

Total **60.0** / 100

Feedback คุณเข้าใจผลกระทบของ shallow copy การเกิด aliasing และการเกิด memory leak ได้เป็นอย่างดี อย่างไรก็ตาม คุณยังสับสนเกี่ยวกับกลไก double delete ในช่วง destruction และขั้นตอนการสร้าง `operator=` รวมถึงยังไม่แม่นยำในเรื่อง Rule of Three และ Copy Constructor แนะนำให้ทบทวนเรื่อง Dynamic Memory Management เพิ่มเติมครับ

Rubric	implicit_assignment_diagnosis	18
	memory_picture_and_leak	18
	consequence_analysis	12
	correct_fix	10
	rule_of_three_generalization	2

Viva Info

Problem Viva: The Missing operator=

User 6730084521

Points 60.0/100

Starts 0 of 1 starts left today

Restart practice viva